

Listing 12.8 Turning a Monoid into an Applicative

```

type Const[M, B] = M

implicit def monoidApplicative[M](M: Monoid[M]) =
  new Applicative[({ type f[x] = Const[M, x] })#f] {
    def unit[A](a: => A): M = M.zero
    def map2[A,B,C](m1: M, m2: M)(f: (A,B) => C): M = M.op(m1,m2)
  }

```

This is `ConstInt` generalized to any `M`, not just `Int`.

This means that `Traverse` can extend `Foldable` and we can give a default implementation of `foldMap` in terms of `traverse`:

```

trait Traverse[F[_]] extends Functor[F] with Foldable[F] {
  ...
  def foldMap[A,M](as: F[A])(f: A => M)(mb: Monoid[M]): M =
    traverse[({ type f[x] = Const[M, x] })#f, A, Nothing](
      as)(f)(monoidApplicative(mb))
}

```

A trait may list multiple supertraits, separated by the keyword `with`.

Scala can't infer the partially applied `Const` type alias here, so we have to provide an annotation.

Note that `Traverse` now extends both `Foldable` *and* `Functor`! Importantly, `Foldable` itself can't extend `Functor`. Even though it's possible to write `map` in terms of a fold for most foldable data structures like `List`, it's not possible *in general*.

**EXERCISE 12.15**

Answer, to your own satisfaction, the question of why it's not possible for `Foldable` to extend `Functor`. Can you think of a `Foldable` that isn't a functor?

So what is `Traverse` really for? We've already seen practical applications of particular instances, such as turning a list of parsers into a parser that produces a list. But in what kinds of cases do we want the *generalization*? What sort of generalized library does `Traverse` allow us to write?

12.7.2 Traversals with State

The `State` applicative functor is a particularly powerful one. Using a `State` action to traverse a collection, we can implement complex traversals that keep some kind of internal state.

An unfortunate amount of type annotation is necessary in order to partially apply `State` in the proper way, but traversing with `State` is common enough that we can create a special method for it and write those type annotations once and for all:

```

def traverseS[S,A,B](fa: F[A])(f: A => State[S, B]): State[S, F[B]] =
  traverse[({ type f[x] = State[S, x] })#f, A, B](fa)(f)(Monad.stateMonad)

```

To demonstrate this, here's a `State` traversal that labels every element with its position. We keep an integer state, starting with 0, and add 1 at each step.

Listing 12.9 Numbering the elements in a traversable

```
def zipWithIndex[A](ta: F[A]): F[(A, Int)] =
  traverseS(ta)((a: A) => (for {
    i <- get[Int]
    _ <- set(i + 1)
  } yield (a, i))).run(0)._1
```

This definition works for `List`, `Tree`, or any other traversable.

Continuing along these lines, we can keep a state of type `List[A]`, to turn any traversable functor into a `List`.

Listing 12.10 Turning traversable functors into lists

```
def toList[A](fa: F[A]): List[A] =
  traverseS(fa)((a: A) => (for {
    as <- get[List[A]]
    _ <- set(a :: as)
  } yield ())).run(Nil)._2.reverse
```

Get the current state,
the accumulated list.

Add the current element and set
the new list as the new state.

We begin with the empty list `Nil` as the initial state, and at every element in the traversal, we add it to the front of the accumulated list. This will of course construct the list in the reverse order of the traversal, so we end by reversing the list that we get from running the completed state action. Note that we `yield ()` because in this instance we don't want to return any value other than the state.

Of course, the code for `toList` and `zipWithIndex` is nearly identical. And in fact most traversals with `State` will follow this exact pattern: we get the current state, compute the next state, set it, and yield some value. We should capture that in a function.

Listing 12.11 Factoring out our `mapAccum` function

```
def mapAccum[S,A,B](fa: F[A], s: S)(f: (A, S) => (B, S)): (F[B], S) =
  traverseS(fa)((a: A) => (for {
    s1 <- get[S]
    (b, s2) = f(a, s1)
    _ <- set(s2)
  } yield b)).run(s)

override def toList[A](fa: F[A]): List[A] =
  mapAccum(fa, List[A]())((a, s) => ((), a :: s))._2.reverse

def zipWithIndex[A](fa: F[A]): F[(A, Int)] =
  mapAccum(fa, 0)((a, s) => ((a, s), s + 1))._1
```